Name: Cohort/Batch: Date: Score:

ELECTRON PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Desktop

TOTAL: 10 QUESTIONS

Q1 What is Electron and what problem in Desktop does it solve? Process Model

Q2 What are the core components or concepts in Electron? Architecture

Q3 How does Electron compare to its main alternatives? Configuration

Q4 How do you get started with Electron for a new project? Hardening

Q5 What configuration options most affect Electron's behavior in production? IPC

Q6 How does Electron handle reliability and high availability? Observability

Q7 What observability does Electron provide out of the box? Lifecycle

Q8 What are common performance bottlenecks in Electron? Compliance

Q9 What security best practices apply when running Electron? Testing

Q10 What mistakes do teams commonly make when adopting Electron? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Electron is a tool or platform that

Mitigation

Electron is a tool or platform that automates or simplifies a specific concern in the Desktop domain, reducing manual effort and operational complexity for engineering teams.

A2 Electron has key building blocks that work

Primitives

Electron has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 Electron trades off simplicity against feature richness

Hardening

Electron trades off simplicity against feature richness differently than its alternatives. Choose Electron when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install Electron via its package manager or

Gotchas

Install Electron via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include concurrency

Hardening

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 Electron provides HA through leader election, replication

Observability

Electron provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 Electron exposes metrics (Prometheus endpoint or built-in

Automation

Electron exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from misconfigured

Performance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run Electron with the principle of least

Assessment

Run Electron with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the documentation and

Organization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.